

Chapter 12 One-Pager — Sequence Models and LLM Agents in Strategic Settings

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Problem. Two post-classical ways to play a game bypass everything Chapters 2-11 built. **Sequence modelling** recasts play as conditional prediction — feed a transformer (**return-to-go**, **state**, **action**) triples and ask for the next action given a desired return, no value function, no self-play — and **ARDT** adds minimax relabelling for worst-case robustness. **LLM agents** skip training entirely: hand a model the rules in English. Chapter 12 scores both against an exact benchmark.

Approach. Kuhn Poker, where Chapter 2 supplies exact Nash and exact exploitability, with a narrow port to Leduc Hold'em as a complexity check. Compared: Nash-CFR, behavioural cloning, a Decision Transformer, ARDT, and four LLM agents (a scripted offline stub with no language model; gpt-oss-20b, Qwen2.5-7B and OpenThinker3-7B, run locally in LM Studio on an RTX 5090 at temperature 0.7), plus follow-ons asking *why* that ranking falls out. **All numbers below are the author's own measurements**, from committed JSON, with two caveats: two results were **retracted** mid-session (an impossible +1.59 gap closed, and a Leduc -0.83 from a decoder discarding 70% of the probability mass), and the Nash reference is 0.0162 chips at 5,000 CFR iterations but 0.0061 at 50,000 — both ~0.

Key results (measured).

- *Plain cloning beats DT and ARDT.* **BC 0.055 < ARDT 0.469 < DT 0.799** against Nash **0.0162**; the real LLMs score **0.25–0.33** (the scripted offline stub: 0.833). Cloning lands within 0.04 chips of Nash by copying a near-Nash policy, while the DT routes that same policy through a return-conditioning channel carrying mostly luck — conditioning actively destroys information here.
- *Return conditioning never steers, on either game.* **high(+2) = 0.6981** vs **low(-2) = 0.6928**: tied, and ordered wrong. The Kuhn explanation (a 4-value payoff alphabet) was **half-refuted** on Leduc: 15 payoff values made the predicted artefact vanish, but steering still failed to appear (**r** = +0.062).
- *The models play better than they can explain.* The hypothesis going in was “knows the right frequency, cannot sample it”; the data says the reverse. Scored as strategies, *stated* frequencies are far worse than *played* ones — Qwen **0.921** vs **0.357** chips, gpt-oss **1.576** vs **0.392**. Probe behaviour; do not ask.
- *Exploitative by default, not adaptive.* Against the one opponent with enough statistical power (AlwaysPass), the model (Qwen2.5-7B, 120 hands) captures **83%** of available exploitation from the first half onward and does not improve; against the other two the error is ±0.53–0.80. Against the bot zoo it wins **0.177 chips/hand** vs **0.110** for Nash at an exploitability of 0.357 vs 0.006, with nearly 90% of the advantage coming from the passive and random opponents.
- *Model size does not explain the ranking, and the LLM edge belongs to Kuhn.* **Qwen2.5-7B beats gpt-oss-20b (3.6B active parameters) by +0.162 chips/hand** over 20k hands per pair, strictly transitively. On Leduc the LLM is indistinguishable from the DT (-0.463 vs -0.454) and loses on average to weak opponents (-0.071 where Nash gets +0.582; it wins only against the two passive types), with **100%** of its illegal-action mass a single misconception: folding when checking is free.
- *One scalar hides the diagnosis.* A single Queen node is **41.4%** of total exploitability, while value-betting the King at 1.00 where the CFR equilibrium mixes at 0.68 costs **0.1%** — deviation size and cost are nearly uncorrelated. Harness: **3 PASS / 2 FAIL**, the failures left as they are.

Thesis connection. Contribution #1 gets a negative result worth having: adaptation from in-context history does not appear (told the opponent's type, the models do respond), so an explicit opponent model must be built rather than assumed. Contribution #2 gets its trade-off measured on one plot — more exploitation (0.177 vs 0.110 chips/hand) at far higher exploitability (0.357 vs 0.006), mostly against weak opposition. Contribution #3 gets the per-decision decomposition and the measurement protocol.

Open questions. The most likely fix is ARDT's $Q(s,a)$ (not yet tested; with chance events such as the deal, ARDT's minimax target is no longer exact): a state-only return estimator cannot tell “this state is bad” from “*this action* is bad”, probably why the tau sweep contradicts theory and exploitability comes out *lowest* on the optimistic side — the default was left at the theoretically correct value rather than switched to buy a better number. Whether the Leduc misconception is one prompt line away. And the standing caveat: every Leduc LLM conclusion rests on one model, one prompt style, 600 hands.